Unit:4 Knowledge Representation

Presented By: Er. Roshan Maharjan

Knowledge:

Knowledge is a theoretical or practical understanding of a subject or a domain. Knowledge is also the sum of what is currently known.

Knowledge is —the sum of what is known: the body of truth, information, and principles acquired by mankind." Or, "Knowledge is what I know, Information is what we know."

There are many other definitions such as:

- Knowledge is "information combined with experience, context, interpretation, and reflection. It is a high-value form of information that is ready to apply to decisions and actions." (T. Davenport et al., 1998)

- Knowledge is —human expertise stored in a person's mind, gained through experience, and interaction with the person's environment." (Sunasee and Sewery, 2002)

- Knowledge is —information evaluated and organized by the human mind so that it can be used purposefully, e.g., conclusions or explanations." (Rousa, 2002)

- Types of knowledge:
  - Classification-based Knowledge: Ability to classify information
  - Decision-oriented Knowledge: Choosing the best option
  - Descriptive knowledge: State of some world (heuristic)
  - Procedural knowledge: How to do something
  - Reasoning knowledge: What conclusion is valid in what situation?
  - Assimilative knowledge: What its impact is?

Knowledge Representation

Knowledge representation (KR) is the study of how knowledge about the world can be represented and what kinds of reasoning can be done with that knowledge. Knowledge Representation is the method used to encode knowledge in Intelligent Systems.

Since knowledge is used to achieve intelligent behavior, the fundamental goal of knowledge representation is to represent knowledge in a manner as to facilitate inferencing (i.e. drawing conclusions) from knowledge. A successful representation of some knowledge must, then, be in a form that is *understandable* by humans, and must cause the system using the knowledge to *behave* as if it knows it.

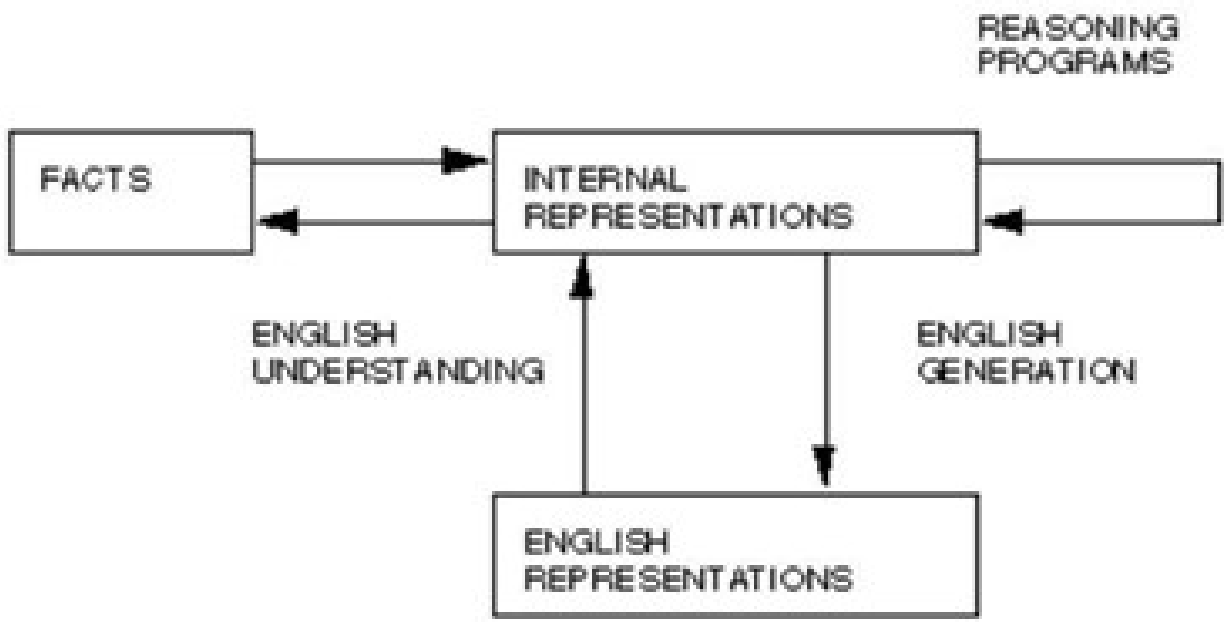


Fig: Two entities in Knowledge Representaion

Knowledge Representation

- The objective of knowledge representation is to express the knowledge about the world in A computer-tractable form.
- This computer- tractable form of knowledge helps the agent to identify patterns of good reasoning and patterns of bad reasoning, so the agent know which to follow and which to avoid.
- A formal language is required to represent knowledge in a computer tractable form and reasoning processes are required to manipulate this knowledge to deduce new facts.
- Key aspects of knowledge representation languages are:
  - Syntax:describeshowsentencesareformedinthelanguage.
  - Semantics: describes the meaning of sentences, what is it the sentence refers to in the real world.
  - Proof Theory (Inference method): Set of rules for generating new sentences that are necessarily true given that the old sentences are true.

- Properties for Knowledge Representation Systems:
  - The following properties should be possessed by a knowledge representation system.
  - Representational Adequacy
    - The ability to represent the required knowledge;
  - Inferential Adequacy
    - The ability to manipulate the knowledge represented to produce new knowledge corresponding to that inferred from the original;
  - Inferential Efficiency
    - the ability to direct the inferential mechanisms into the most productive directions by storing appropriate guides;
  - Acquisitional Efficiency
    - the ability to acquire new knowledge using automatic methods wherever possible rather than reliance on human intervention.

Issues in Knowledge Representation

- Are any attributes of objects so basic that they have been occurred in almost every problem domain?
- Are there any important relationships that exist among attributes of objects?
- At what level should knowledge be represented?
- How should sets of objects be represented?
- How can relevant part be accessed when they are needed?

1. Semantic Nets

Semantic network is an alternative to predicate logic as a form of knowledge representation.

The structure of a semantic net is shown graphically in terms of nodes and the arcs connecting them.

.Nodes is sometimes referred to as objects.
.Arcs represents the links or edges.
.The links are used to express relationships.

Two types of commonly used links are

i. IS-A, and
ii. A-KIND-OF

IS-A means "is an instance of' and refers to a specific member of a class.

The link A-KIND-OF is used here to relate one class to another.

Nodes are to represent physical objects, concepts, or situation.

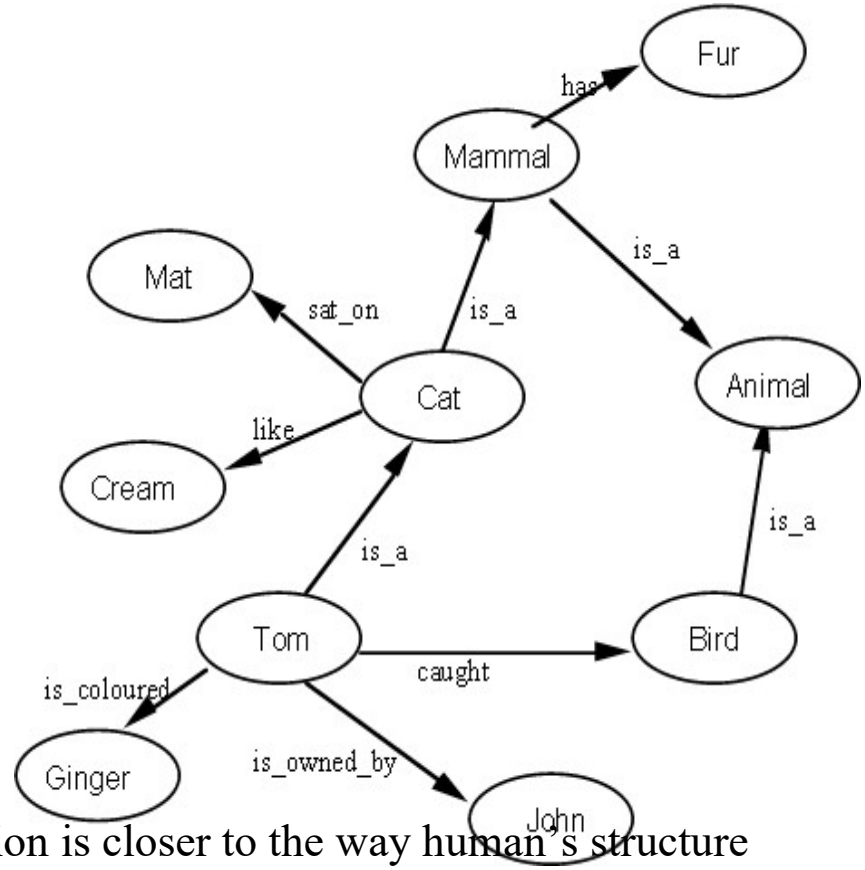
The idea is that we can store our knowledge in the form of a graph, with nodes representing objects in the world, and arcs representing relationships between those objects.

Semantic network is a declarative graphic representation that can be used to represent

Knowledge and support automated systems for reasoning about the knowledge

For example, the following:

-
- Semantic Network is intended to represent the data:
- Tom is a cat.
- Tom caught a bird.
- Tom is owned by John.
- Tom is ginger in colour.
- Cats like cream.
- The cat sat on the mat.
- A cat is a mammal.
- A bird is an animal.
- All mammals are animals.
- Mammals have fur.
- It is argued that this form of representation is closer to the way human's structure knowledge.



Example

(1) A trout is a fish.

(2) A fish has gills.

(3) A fish has fins.

(4) Fish is food.

(5) Fish is animal.

(6) An apple is a fruit.

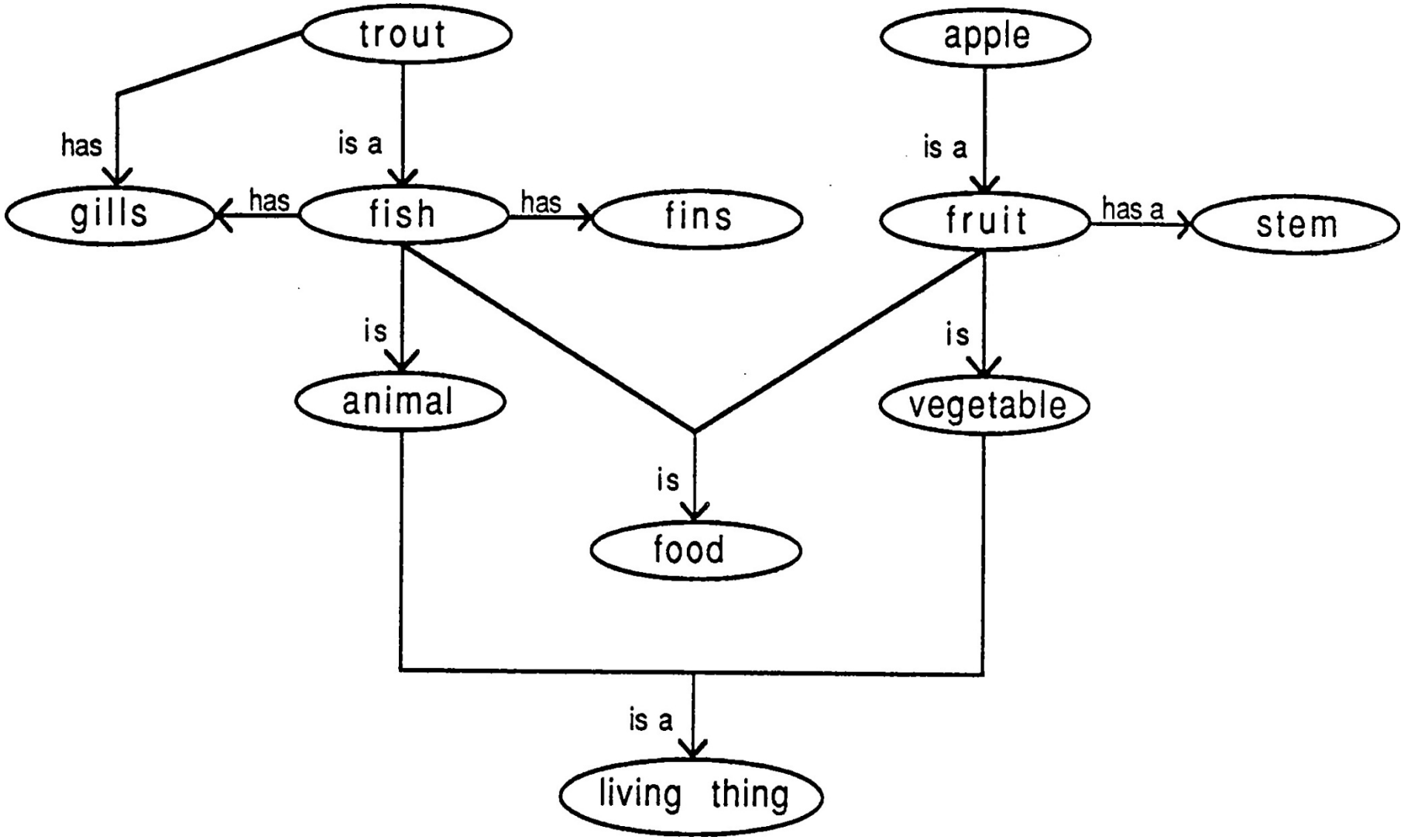
(7) Fruit has a stem.

(8) Fruit is food.

(9) Fruit is vegetable.

(10) An animal is a living thing.

(11) A vegetable is a living thing.


trout
apple
has
is a
is a
gills
has
fish
has
fins
fruit
has a
stem
is
is
animal
vegetable
is
food
is a
living thing

Frames (slot and filler structure)

- Frame is a type of schema used in many AI applications including vision and natural language processing that provides a convenient structure for representing.
- Frame is similar to a record structure and corresponding to the fields and values are slots and slot fillers.
- Basically it is a group of slots and fillers that defines a stereotypical object.
- A single frame is not much useful. Frame systems usually have collection of frames connected to each other.
- Value of an attribute of one frame may be another frame.
- Frames are also useful for representing common sense knowledge.
- While semantic nets are basically a two-dimensional representation of knowledge, frames add a third dimension by allowing nodes to have structures.
- By using frames in the filler slots and inheritance, very powerful knowledge
- Representation systems can be built.
- Frame-based expert systems are very useful for representing causal knowledge because their information is organized by cause and effect.
- Frames are generally designed to represent either generic or specific knowledge.

- A frame for a book is given below.

Slots	Fillers
publisher	Thomson
title	Expert b
author	Giarratano
edition	Third
year	1998
pages	600

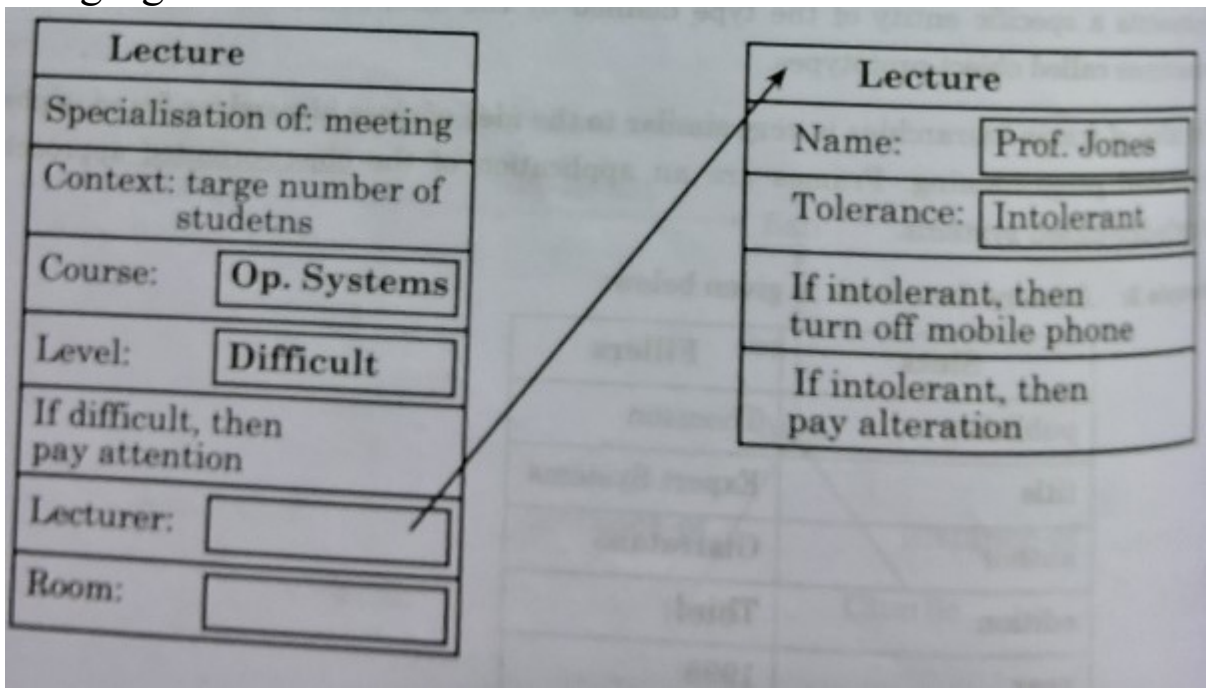
-

Example of generic frame

Slot	Fillers
name	computer
specialization_of	a_kind_of machine
types	(desktop, laptop, mainframe, super) if-added: Procedure ADD_COMPUTER
speed	default: faster if-needed: Procedure FIND_SPEED
location	(home, office, mobile)
under_warranty	(yes, no)

-

The following fig shows the two frames lecture and lecturer connected with links



-

Conceptual Dependency (CD) (Strong slot and filler structure)

- Conceptual Dependency originally developed to represent knowledge acquired from natural language input.
- The goals of this theory are:
  - To help in the drawing of inference from sentences.
  - To be independent of the words used in the original input.

- For any 2 (or more) sentences that are identical in meaning there should be only one representation of that meaning.
- It has been used by many programs that portend to understand English.
- CD provides:
  - A structure into which nodes representing information can be placed
  - specific set of primitives
  - At a given level of granularity.

- Sentences are represented as a series of diagrams depicting actions using both abstract and real physical situations.
- The agent and the objects are represented.
- The actions are built up from a set of primitive acts which can be modified by tense.

Examples of Primitive Acts are:

ATRANS
--Transfer of an abstract relationship. e.g. give.

PTRANS
--Transfer of the physical location of an object. e.g. go.

PROPEL
--Application of a physical force to an object. e.g. push.

MTRANS
--Transfer of mental information .e.g. tell.

MBUILD
--Construct new information from old. e.g. decide.

SPEAK
--Utter a sound. e.g. say.

ATTEND
--Focus a sense on a stimulus. e.g. listen, watch.

MOVE
--Movement of a body part by owner .e.g. punch, kick.

GRASP
--Actor grasping an object. e.g. clutch.

INGEST
--Actor ingesting an object. e.g.eat.

EXPEL
--Actor getting rid of an object from body. e.g. throw

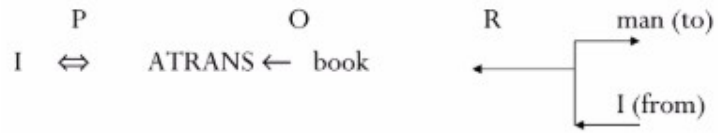
Conceptual Category:

- There are four conceptual categories

  - ACT — Actions {one of the CD primitives}
  - PP — Objects {picture producers}
  - AA — Modifiers of actions {action aiders}
  - PA — Modifiers of PP's {picture aiders}

Example:

- I gave a book to the man. CD representation is as follows:



- It should be noted that this representation is same for different saying with same meaning. For example
  - I gave the man a book,
  - The man got book from me,
  - The book was given to man by me etc.

Few Conventions:

- Arrows indicate directions of dependency
- Double arrow indicates two way link between actor and action.

  O – for the object case relation

  R – for the recipient case relation

  P – for past tense

  D - destination

Rule 1: PP ⇔ ACT

- It describes the relationship between an actor and the event he or she causes.
  - This is a two-way dependency, since neither actor nor event can be considered primary.
  - The letter P in the dependency link indicates past tense.
- Example: John ran

CD Rep: John $\overset{P}{\Leftrightarrow}$ PTRANS

Rule 2: ACT ← PP

- It describes the relationship between a ACT and a PP (object) of ACT.
  - The direction of the arrow is toward the ACT since the context of the specific ACT determines the meaning of the object relation.
- Example: John pushed the bike

CD Rep: John ⇔ PROPEL $\overset{o}{\leftarrow}$ bike

Rule 3: PP ↔ PP

- It describes the relationship between two PP's, one of which belongs to the set defined by the other.
- Example: John is doctor

CD Rep: John ↔ doctor

Rule 4: PP ← PP

- It describes the relationship between two PP's, one of which provides a particular kind of information about the other.
  - The three most common types of information to be provided in this way are possession (shown as POSS-BY), location (shown as LOC), and physical containment (shown as CONT).
  - The direction of the arrow is again toward the concept being described.
- Example: John's dog

poss-by

Rule 5: PP ⇔ PA

- It describes the relationship between a PP and a PA that is asserted to describe it.
  - PA represents states of PP such as height, health etc.

- Example: John is fat

CD Rep John ⇔ weight (> 80)

Rule 6: PP ← PA

- It describes the relationship between a PP and an attribute that already has been predicated of it.
  - Direction is towards PP being described.

- Example: Smart John

CD Rep John ← smart

Rule 7: ACT ← R
→ PP (to)
← PP (from)

- It describes the relationship between an ACT and the source and the recipient of the ACT

- Example: John took the book from Mary

CD Rep: John ⇔ ATRANS←R
o ↑
book
→ John
← Mary

Rule 8: PP ← PA

→ PA
← PA

- It describes the relationship that describes the change in state.

- Example: Tree grows

CD Rep: Tree ←
→ size > C
← size = C

Scripts

- Scripts were introduced by Schank and Abelson introduced in 1977 that used CD framework.
- The scripts are useful in describing certain stereotyped situations such as going to theater
- It consists of set of slots containing default values along with some information about the type of values similar to frames.
- It differs from FS as the values of the slots in scripts must be ordered and have more specialized roles.
- In real world situations, we see that event tends to occur in known patterns because of clausal relationship to the occurrence of events

Script Components:

- Each script contains the following main components.
  - ***Entry Conditions:*** Must be satisfied before events in the script can occur.
  - ***Results:*** Conditions that will be true after events in script occur.
  - ***Props:*** Slots representing objects involved in the events.
  - ***Roles:*** Persons involved in the events.
  - ***Track:*** Specific variation on more general pattern in the script. Different tracks may share many components of the same script but not all.
  - ***Scenes:*** The sequence of events that occur. Events are represented in conceptual dependency form.

Example

Script : Play in theater	Various Scenes
Track: Play in Theater **Props:** • Tickets • Seat • Play **Roles:** • Person (who wants to see a play) – P • Ticket distributor – TD • Ticket checker – TC **Entry Conditions:** • P wants to see a play • P has a money **Results:** • P saw a play • P has less money • P is happy (optional if he liked the play)	***Scene 1: Going to theater*** • P PTRANS P into theater • P ATTEND eyes to ticket counter ***Scene 2: Buying ticket*** • P PTRANS P to ticket counter • P MTRANS (need a ticket) to TD • TD ATRANS ticket to P ***Scene 3: Going inside hall of theater and sitting on a seat*** • P PTRANS P into Hall of theater • TC ATTEND eyes on ticket POSS_by P • TC MTRANS (showed seat) to P • P PTRANS P to seat • P MOVES P to sitting position ***Scene 4: Watching a play*** • P ATTEND eyes on play • P MBUILD (good moments) from play ***Scene5: Exiting*** • P PTRANS P out of Hall and theater

A Restaurant Script

Script: RESTAURANT	
Track: Coffee Shop Props: Tables Menu F=Food Check Money Roles S=Customer W=Waiter C=Cook M=Cashier O=Owner	Entry cond.: S hungry S has money Results: S has less money O has more money S is not hungry S is pleased (optional)
Scene 1: Entering S PTRANS S into restaurant S ATTEND eyes to tables S MBUILD where to sit S PTRANS S to table S MOVE S to sitting position	**Scene 2: Order** (Menu on table) (W brings menu) S PTRANS menu to S S MTRANS food list to S S MTRANS signal to W W PTRANS W to table S MTRANS 'I want F' to W
Scene 3: Eating C ATRANS F to W W ATRANS F to S S INGEST F (Option: Return to Scene 2 to order more; otherwise, go to Scene 4)	**Scene 4: Exiting** W MOVE (write check) W PTRANS W to S W ATRANS check to S S ATRANS tip to S S PTRANS S to M S ATRANS money to M S PTRANS S to out of restaurant

Rule Based Knowledge Representation System (RBS)

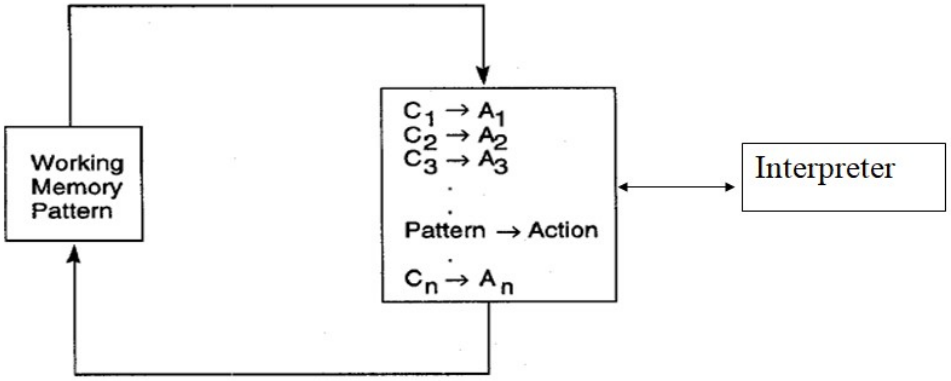
- A rule-based system (or production system) is a knowledge based system (KBS) in which the knowledge is stored as rules; an expert system is RBSs in which the rules come from human experts in a particular domain.

- The underlying idea of production systems is to represent knowledge in the form of condition-action pairs called production rules.

  - If condition C is satisfied, then action A is appropriate.

- The syntax of rules is:

  - IF<premise> THEN<action>



- ## Components of RBS
  - **Working memory:** a small allocation of memory into which only appropriate rules are copied
  - **Rule base:** the rules themselves, possibly stored specially to facilitate efficient access to the antecedent
  - **Interpreter:** The processing engine which carries out reasoning on the rules and derives answer.
  - A user interface or other connection outside world through which input and output signals are received and sent

The Interpreter cycle

- The execution of a production system can be defined as recognize-act cycle:
- Match-memory contains matched against the condition of production rules, this produces a subset of production called conflict-set.
- Conflict resolution- one of the production in the conflict set is then selected
- Apply the rule.

Problem: sorting a string composed of the letters a, b, and c.

- Working memory: cbaca
- Production set:

1. ba → ab
2. ca → ac
3. cb → bc

- In this example, a production is enabled if its condition matches a portion of the string in working memory.
- When a rule is fired, the substring that matched the rule condition is replaced by the string on the right hand side of the rule.

Iteration #	Working memory	Conflict set	Rule fired
0	cbaca	1, 2, 3	1
1	cabca	2	2
2	acbca	3, 2	2
3	acbac	1, 3	1
4	acabc	2	2
5	aacbc	3	3
6	aabcc	Ø	Halt

Fig: Trace of simple production system